

100 points possible.

Name: _____

***Submit your pdf answer file and two code files, your Ruby and Python codes.**

Part1-Ruby: 50 pts

1. (12 pts) Write a Ruby regular expression that would match each of the following types of strings.

a) (4 pts) Strings that contain exactly two integer numbers (each number may have any number of digits). The string may contain other words/characters as well.

(Examples: "There were 20 dogs in 5 cars." Or "There is 1 barn on the 237 acres")

*Add your code to the Ruby file.

b) (4 pts) Strings that contain no numeric (digit) characters.

*Add your code to the Ruby file.

c) (4 pts) Strings that start with the letter "s", followed by two vowels, and that have the same two consecutive vowels later in the string: (for example: "sourdough").

*Add your code to the Ruby file.

2. (7 pts) Suppose I have an object of class Automobile called car1, and I call the method car1.horsepower(). Describe the process that Ruby uses to search for the horsepower() method. What happens if there is no such method? How does ruby ensure that mixins will be searched in this process?

3. (8 pts) Could you write a Ruby-style *iterator* in Racket? Why or why not? (*Hint: first define what exactly an iterator is in Ruby. Once you have done this, ask yourself “Could I create one of those in Racket?” If the answer is yes, give an example or show how. If the answer is no, what is Racket missing that you would need to do it?*)
4. (7 pts) What are *dynamic methods* in Ruby? Give two examples of situations in which dynamic methods are useful.

5. (9 pts) Write a Ruby mixin called *Countable* that adds the method *count* to any class. The *count* method should return the number of items enumerated by the *each* method. If included in a class that does not provide *each*, calling *count* should return zero.

*Add the code to the Ruby file

6. (7 pts) There are two main ways in which computer scientists describe the variable-typing policies of languages: strong vs. weak, and dynamic vs. static

Give an example of a strongly-typed language and a weakly-typed language and describe what it is about the language that makes it strongly or weakly typed.

Part2-Python: 50 pts

7. (3 pts) In Tri-color algorithm every object in the system is “marked” in one of three colors, White, Gray, or Black. In your own words, describe the Tri-color algorithm, its initialization and the tracing steps, and then color each object in the system.

I-Tri-color algorithm:

The general idea is....

White:

Gray:

Black:

II-Initialization:

Before the algorithm starts...

White:

Gray:

Black:

III-Describe the Tracing steps:

After the Tracing is done:

White:

Gray:

Black:

8. (6 pts) Suppose you are given the following segment of C++ code:

```
int global1;
int *global2;

int sub1 (int x) {
    int sub1var1;
    int *sub1ptr1;

    sub1ptr1 = new int;
    ... more code ...
    return sub1var1;
}

int main () {
    int mainvar1, mainvar2;
    float mainvar3;
    int *mainptr1;
    double *mainptr2;

    mainptr1 = new int;
    mainptr2 = new double;
    mainvar1 = sub1(mainvar2);
    ... more code ...
    return;
}
```

- a- (2 pts) If the program is paused for garbage collection, using a conservative mark-and-sweep garbage collection algorithm, while it is in `main()` just before calling `sub1()`, what set (black, white, or gray) will each object in the program be in when garbage collection starts? For pointers, include both the pointer itself and the object/data it is pointing to.

White:

Gray:

Black:

- b- (2 pts) Suppose that the garbage collector begins the tri-color algorithm by processing every item in the root set first. After all of the roots are processed, what set will each of the variables belong to?

White:

Gray:

Black:

c- (2 pts) What if the program is paused for garbage collection while in `sub1()`, just before executing the return statement? At the start of GC, what set will each variable belong to?

White:

Gray:

Black:

9. (3 pts) Write a short (a line or two) Python program that demonstrates that Python is introspective. Write your short answer here and add any example code to the Python file:
10. (3 pts) Write a short (a line or two) Python program that demonstrates that integer variables are actually objects. Write your short answer here and add any example code to the Python file:
11. (3 pts) Write a short (a few lines) Python program that demonstrates that functions are first-class objects in Python. Write your short answer here and add any example code to the Python file:

12. (3 pts) Write a short answer (a few lines) to explain what is a destructive method. What is the difference between `sort()` and `sorted()` (show examples). Write your short answer here and add any example code to the Python file:
13. (3 pts) Write a short answer (a few lines) to explain what is list comprehension in Python and give an example with multiple conditions. Write your short answer here and add any example code to the Python file:
14. (3 pts) Write a short answer (a few lines) to explain what is an iterator and what is an iterable object. Name two examples of iterables in Python. Write your short answer here and add any example code to the Python file:
15. (3 pts) Write a short answer (a few lines) to explain what is a generator in Python and how it is different from an iterator. Write your short answer here and add any example code to the Python file:

16. (3 pts) Write a short answer (a few lines) to explain what is a decorator in Python and how it is used (show with an example). Write your short answer here and add any example code to the Python file:

17. (8 pts) Write a Python generator that produces binary numbers as strings. In other words, if your generator is called `my_generator`, the following code:

```
for val in my_generator():  
    print(val)  
    if (val == '1000'):  
        break
```

Should print the sequence: 0, 1, 10, 11, 100, 101, 110, 111, 1000

*Add the code to the Python file.

18. (9 pts) Write a Python decorator, `print_instead` that changes a function that returns a single value to instead print that value and return None. If the following simple recursive function that computes the n^{th} Fibonacci number, were decorated with `print_instead`, as follows:


```
@print_instead
def fibonacci(n):
    current = 1
    previous = 1
    second_previous = 0
    if (n <= 0):
        return 0

    for _ in range(n-2):
        second_previous = previous
        previous = current
        current = previous + second_previous

    return current
```

Then executing the line:

```
result = fibonacci(7)
```

should place the value **None** into **result** and print the value **13** to the screen.

Your decorator should work on any function, including functions that take any number (including zero) of positional and/or keyword arguments.

*Add the code to the Python file.